Train-Test Split Concept

Before building any machine learning model, we must answer one important question:

> Has the model really learned, or has it only memorized?
> Train-test split helps us check that.

Simple Meaning

Train-test split divides data into two parts:

Training Data	Testing Data
Used to teach the model — **80%** of the data	Used to check the model — **20%** of the data

ⓘ **Golden Rule:** Do not test the model on the same data it learned from.

The Student Analogy

A teacher gives 100 questions. If the student studies *all* 100 and the exam uses the *same* 100, a high score proves memory — not understanding.

80 Questions

For practice (training)

20 New Questions

For the exam (testing)

Machine learning works the same way.

ML Analogy

Training Data = Practice Questions

The model learns patterns from training data.

Testing Data = Exam Questions

We check the model using testing data. Good performance on testing data means the model can handle new, unseen data.

House Price Example

Goal: Predict house price from features like area, bedrooms, age, and distance.

Area	Bedrooms	Age	Distance	Price
1000	2	5	8	55
1500	3	3	5	82
2000	4	2	4	120
2500	4	1	2	155
2800	5	1	2	175

10 total rows — we split into 8 for training and 2 for testing.

How We Split the Data

The model learns from 8 training rows. The prices of the 2 testing rows are hidden, and the model must predict them. We then compare actual price vs predicted price.

Main Goal

Train-test split checks: Can the model perform well on new, unseen data?

New House Price

Student Result

Loan Approval

Sales Forecast

What Happens Without Train-Test Split?

Wrong Approach
Train on all data, then test on the same data. Results look great — but it's misleading.

The model already saw the answers. Like a student who gets the exam paper in advance — the score may be high, but understanding is low.

Correct Approach
Keep testing data separate. Never let the model see it during training.

Only then can we trust that good performance reflects genuine learning.

Overfitting

> The model memorizes training data too much and performs poorly on new data.

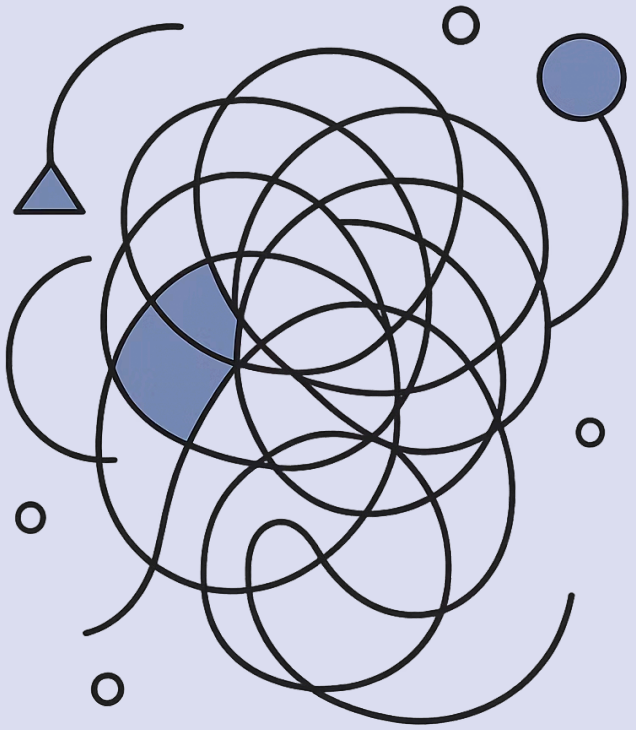
A student memorizes: 2+2=4, 3+3=6, 4+4=8 — but when asked 5+5=?, gets confused. They memorized examples without understanding the concept.

Training Performance

Very high ✅

Testing Performance

Poor ❌



Underfitting

Underfitting means the model has not learned enough even from the training data.

Training Performance

Poor ❌ — like a student who didn't study

Testing Performance

Also poor ❌ — fails on both practice and exam

ⓘ Underfitting = model too simple. Overfitting = model too complex. We aim for the balance in between.

Good Model Behavior

> A good model performs reasonably well on both training data and testing data.

Training Performance

Good ✅

Testing Performance

Also Good ✅

This means the model learned the pattern — not just memorized examples.

Train-Test Split in Python

We use `train_test_split` from scikit-learn:

```
from sklearn.model_selection import train_test_split

X = df[["area", "bedrooms", "age", "distance"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Separate features (`X`) and target (`y`), then split with 20% for testing.

What These Variables Mean

Variable	Meaning
`X_train`	Input features for training
`X_test`	Input features for testing
`y_train`	Correct answers for training
`y_test`	Correct answers for testing

Training	Testing
`X_train` + `y_train`	`X_test` + `y_test`

Model Flow

`y_test` = actual answers | `y_pred` = model predictions. Comparing these two tells us how well the model performs.

Meaning of `test_size` and `random_state`

`test_size=0.2`

20% of data used for testing, 80% for training.

Example with 1,000 rows:

- Training rows: 800
- Testing rows: 200

`random_state=42`

Fixes the randomness so we get the same split every time.

Without it, every run may create a different split — making results hard to reproduce.

Full Python Example

```
import pandas as pd
from sklearn.model_selection import train_test_split

data = {
 "area": [1000,1500,2000,900,1800,2200,1300,2500,1600,2800],
 "bedrooms": [2,3,4,2,3,4,2,4,3,5],
 "age": [5,3,2,10,4,2,6,1,5,1],
 "distance": [8,5,4,12,6,3,9,2,7,2],
 "price": [55,82,120,42,95,130,65,155,78,175]
}
df = pd.DataFrame(data)
X = df[["area","bedrooms","age","distance"]]
y = df["price"]

X_train,X_test,y_train,y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)
print("Total rows:", len(df))
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

Expected Output

```
Total rows: 10
Training rows: 8
Testing rows: 2
```

10	8	2
Total Records	Training Rows	Testing Rows
Full dataset	Model learns from these	Unseen during training

Visual Understanding

Before Split

Full dataset — 100%

After Split

80% Training Data — the model sees this while learning.

20% Testing Data — kept aside, never seen during training.

Important Mistake to Avoid

❌ Wrong

```
model.fit(X, y)
y_pred = model.predict(X)
```

Model tested on data it already saw — results are misleading.

✅ Correct

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Model tested on unseen data — results are trustworthy.

> Golden Rule: Train on training data. Test on testing data.

Real-Life Banking Example

A bank builds a loan approval model using past customer data:

01

Features

Income, credit score, loan amount, employment status, default history

02

Split Data

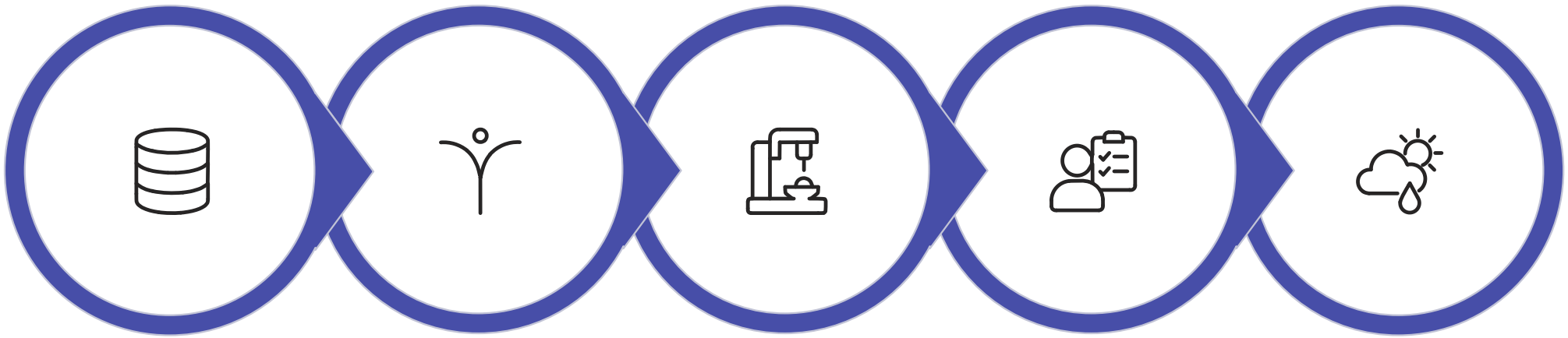
Old records → training data for learning + testing data for checking

03

Deploy

If testing performance is good, use the model for new customers

Train-Test Split and Future Prediction



Example: A new customer applies for a loan. The trained model predicts: Approved or Rejected.

Final Summary

Train-test split checks whether the model can work on unseen data.

1

Split

Divide full data into train and test

2

Train

Model learns from training data

3

Predict

Model predicts on testing data

4

Compare

Prediction vs actual answer

The Golden Rule

Training Data

For learning — the model studies this

Testing Data

For checking — the model is evaluated on this

⊗ Never test the model only on the same data it learned from. That creates fake confidence.

Quick Quiz

> A company has 10,000 old customer records. They use 8,000 to train the model and 2,000 to check it. What is this called?

Answer

Train-Test Split — data is divided into training data (80%) and testing data (20%).

Key Takeaways

Split your data

80% training, 20% testing is the standard starting point

Avoid overfitting & underfitting

A good model performs well on *both* training and testing data

Use `random_state`

Ensures reproducible, consistent splits every run

Train on train. Test on test.

This is the golden rule of machine learning evaluation